

扩散模型基础

从加噪到生成：DDPM 的直观理解、下界推导与采样

2026 年 9 月

本文是扩散模型的学习笔记与讲解稿模版。章节的组织方式参考了邱锡鹏老师《神经网络与深度学习》的写法——概念先行、直觉在前、公式在后；讲解的脉络则参考了李宏毅（Hung-yi Lee）老师的 DDPM 系列课件，尤其是“VAE 与扩散模型的对照”和“训练时想像中与实际上”两处。文中附上了笔者自己的推导与理解，如有错误或问题，欢迎指正与交流！

1 生成模型与扩散模型的基本概念

1.1 生成模型要解决什么问题

在监督学习里，我们关心的是从输入到输出的映射：给定样本 $(\mathbf{x}, y)$ ，学一个函数 f ，让它在新样本上预测得准。这是判别的思路，模型刻画的是条件分布 $p(y | \mathbf{x})$ 。生成模型问的是另一个问题：数据本身服从什么分布？

记真实数据分布为 $p_{\text{data}}(\mathbf{x})$ 。手头只有从它当中独立抽取的 N 个样本 $\{\mathbf{x}^{(n)}\}_{n=1}^N$ （一堆图片、一段语音、一句文本），而 p_{data} 的具体形式是未知的。我们希望学到一个模型 $p_{\theta}(\mathbf{x})$ ，使得从 p_{θ} 采样出来的东西与训练集里的东西无法区分。按最大似然估计，

$$\begin{aligned}\theta^* &= \arg \max_{\theta} \prod_{n=1}^N p_{\theta}(\mathbf{x}^{(n)}) = \arg \max_{\theta} \sum_{n=1}^N \log p_{\theta}(\mathbf{x}^{(n)}) \\ &\approx \arg \max_{\theta} \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} [\log p_{\theta}(\mathbf{x})].\end{aligned}\tag{1}$$

把期望写成积分，再减掉一个与 θ 无关的常数项：

$$\begin{aligned}\arg \max_{\theta} \mathbb{E}_{p_{\text{data}}} [\log p_{\theta}(\mathbf{x})] &= \arg \max_{\theta} \int_{\mathbf{x}} p_{\text{data}}(\mathbf{x}) \log p_{\theta}(\mathbf{x}) d\mathbf{x} \\ &\quad - \underbrace{\int_{\mathbf{x}} p_{\text{data}}(\mathbf{x}) \log p_{\text{data}}(\mathbf{x}) d\mathbf{x}}_{\text{常数}} \\ &= \arg \min_{\theta} \int_{\mathbf{x}} p_{\text{data}}(\mathbf{x}) \log \frac{p_{\text{data}}(\mathbf{x})}{p_{\theta}(\mathbf{x})} d\mathbf{x} \\ &= \arg \min_{\theta} D_{\text{KL}}(p_{\text{data}} \parallel p_{\theta}).\end{aligned}\tag{2}$$

也就是说，最大似然等价于最小化真实分布与模型分布之间的 KL 散度 [13]。这条结论把“训好一个生成模型”还原成了一个统计问题，也是后面所有推导的出发点。

问题在于 $p_\theta(\mathbf{x})$ 本身通常算不出来：它往往是某个隐变量积分掉之后的结果，积分不可解。所有生成模型的差别，就落在“如何绕开这个积分”上。一个统一的写法是：先取一个简单、易采样的先验分布 $\mathbf{z} \sim p(\mathbf{z})$ （通常取标准高斯），再用一个网络 G_θ 把它变换成数据

$$\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \quad \mathbf{x} = G_\theta(\mathbf{z}), \quad (3)$$

差别只在 G_θ 的结构与训练目标。表 1 给出了几条主要路线的对比。

表 1: 几类主流生成模型的对比

模型	生成方式	似然	训练	特点
GAN	一步映射 $G_\theta(\mathbf{z})$	不可计算	对抗博弈，易不稳定	快，但易模式塌缩
VAE	一步映射 + 隐变量	下界	稳定	快，但结果偏模糊
流模型	可逆映射， $\mathbf{z} \leftrightarrow \mathbf{x}$	精确	稳定	快，但受维数与可逆性约束
扩散模型	从噪声出发迭代去噪 T 次	下界	稳定	慢，但质量高、多样性强

1.2 扩散模型的两个过程

扩散模型的做法是把式 (3) 中“一步到位”的映射拆成许多步 [3, 9]，它包含一正一反两个过程（图 1）：

- **前向过程（加噪）**：预先设定、无参数。从干净数据 $\mathbf{x}_0$ 出发，每一步叠加上一小撮高斯噪声，共走 T 步，最终 $\mathbf{x}_T$ 几乎变成纯噪声，分布收敛到 $\mathcal{N}(\mathbf{0}, \mathbf{I})$ ；
- **反向过程（去噪）**：需要学习。从 $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 出发，每一步让网络预测“这一步大概被加进了什么噪声”并把它减掉，逐步还原出 $\mathbf{x}_0$ 。

值得强调的是：前向过程每一步都在丢失信息，它是一个不可逆的物理过程，就像一滴墨水滴进清水后逐渐扩散均匀、熵在增加。我们并不去求它的数学逆，而是让网络学一个统计意义上的逆——给定当前含噪样本，估计这一步被

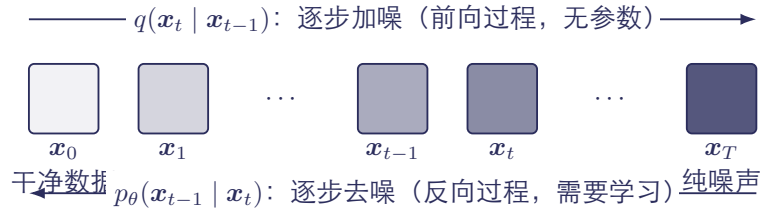


图 1: 扩散模型的前向过程与反向过程。方块由浅到深表示噪声逐渐增大

加入的噪声，然后减掉它。这也正是 DDPM 全名的由来：Denoising Diffusion Probabilistic Models，去噪扩散概率模型 [3]。

换一个角度，扩散模型是一种自回归式的生成：把“一次到位”改成“ N 次到位”[13]。GAN 与 VAE 一步就把 z 变成 x ，扩散模型则用 T 次微小的、几乎确定的去噪步骤完成同一件事。代价是采样变慢（需要 T 次网络前向），换来的是训练的稳定与生成质量的提升。

值得注意的是，这里的“去噪”每一步都只处理一点点噪声，于是每一步的子任务都极其简单，用一个小网络就能拟合得很好；把 T 步串起来，整体就成了一个表达能力极强的生成器。这与“把难题拆成一串简单题”的通用思路是一致的。

1.3 记号约定

后面会反复出现下列记号，先集中列在这里，读时可随时回查。

- $\mathbf{x}_0 \in \mathbb{R}^d$: 干净数据 ($\mathbf{x}_0 \sim p_{\text{data}}$);
- $\mathbf{x}_1, \dots, \mathbf{x}_T \in \mathbb{R}^d$: 逐步含噪的隐变量，与数据同维（这一点与 VAE 不同）;
- T : 扩散步数，常见 200 ~ 1000;
- $\beta_t \in (0, 1)$: 第 t 步的噪声方差; $\{\beta_1, \dots, \beta_T\}$ 称为噪声调度 (noise schedule);
- $\alpha_t := 1 - \beta_t$, $\bar{\alpha}_t := \prod_{s=1}^t \alpha_s$ (注意是累乘);
- $\varepsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$: 标准高斯噪声;
- θ : 网络参数; $\varepsilon_{\theta}(\mathbf{x}_t, t)$ 表示噪声预测网络的输出。

表 2 是本笔记的一个“地图”：扩散模型里出现的所有分布都是高斯，整篇推导其实就是在算它们的均值与方差。

表 2: 四个核心分布：全部都是高斯

分布	均值	方差
前向单步 $q(\mathbf{x}_t \mathbf{x}_{t-1})$	$\sqrt{\alpha_t} \mathbf{x}_{t-1}$	$\beta_t \mathbf{I}$
前向闭式 $q(\mathbf{x}_t \mathbf{x}_0)$	$\sqrt{\bar{\alpha}_t} \mathbf{x}_0$	$(1 - \bar{\alpha}_t) \mathbf{I}$
真实后验 $q(\mathbf{x}_{t-1} \mathbf{x}_t, \mathbf{x}_0)$	$\frac{\sqrt{\bar{\alpha}_{t-1}} \beta_t \mathbf{x}_0 + \sqrt{\alpha_t} (1 - \bar{\alpha}_{t-1}) \mathbf{x}_t}{1 - \bar{\alpha}_t}$	$\frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t \mathbf{I}$
反向过程 $p_\theta(\mathbf{x}_{t-1} \mathbf{x}_t)$	$\mu_\theta(\mathbf{x}_t, t)$ (网络给出)	$\sigma_t^2 \mathbf{I}$ (可取 β_t 或 $\tilde{\beta}_t$)

2 前向扩散过程

2.1 单步加噪

定义 2.1 (前向扩散过程) 给定噪声调度 $\{\beta_1, \dots, \beta_T\}$, $\beta_t \in (0, 1)$, 前向过程按如下马尔可夫链逐步给数据加噪:

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I}), \quad t = 1, 2, \dots, T, \quad (4)$$

等价地写成重参数化的形式

$$\mathbf{x}_t = \sqrt{1 - \beta_t} \mathbf{x}_{t-1} + \sqrt{\beta_t} \boldsymbol{\varepsilon}_{t-1}, \quad \boldsymbol{\varepsilon}_{t-1} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \text{ 相互独立}. \quad (5)$$

记 $\alpha_t = 1 - \beta_t$, 则整条链的联合分布为 $q(\mathbf{x}_{1:T} | \mathbf{x}_0) = \prod_{t=1}^T q(\mathbf{x}_t | \mathbf{x}_{t-1})$ 。

注 2.2 (为什么系数是 $\sqrt{1 - \beta_t}$ 而不是 1) 式 (5) 里信号的系数与噪声的系数是一对平方和为 1 的数: $(\sqrt{1 - \beta_t})^2 + (\sqrt{\beta_t})^2 = 1$ 。若 $\mathbf{x}_{t-1}$ 的各维独立、均值为 0、方差为 1, 那么 $\mathbf{x}_t$ 的方差为 $(1 - \beta_t) \cdot 1 + \beta_t = 1$, 方差始终为 1。这个约定称为保方差 (variance preserving)。反之若写成 $\mathbf{x}_t = \mathbf{x}_{t-1} + \sqrt{\beta_t} \boldsymbol{\varepsilon}$, 方差会随 t 线性增长, 到 $T = 1000$ 时信号会被彻底淹没, 网络也就学不动了。

2.2 任意时刻的闭式解

逐层递推 T 次显然不可行。但幸运的是, 由于每一步都是高斯, 整条链可以被一步折叠, 直接写出任意 t 时刻 $\mathbf{x}_t$ 关于 $\mathbf{x}_0$ 的分布——这是扩散模型最核心的一条性质。

命题 2.3 (前向过程的闭式解) 对任意 $t = 1, \dots, T$,

$$q(\mathbf{x}_t | \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I}), \quad (6)$$

即

$$\boxed{\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\varepsilon}, \quad \boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}).} \quad (7)$$

证明 对 t 归纳。 $t = 1$ 时即为定义式 (5)。设 $t - 1$ 时命题成立, 即 $\mathbf{x}_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_{t-1}} \boldsymbol{\varepsilon}'$, 代入单步公式并合并同类项:

$$\mathbf{x}_t = \sqrt{\alpha_t} \mathbf{x}_{t-1} + \sqrt{\beta_t} \boldsymbol{\varepsilon}'' = \sqrt{\alpha_t \bar{\alpha}_{t-1}} \mathbf{x}_0 + \underbrace{\sqrt{\alpha_t (1 - \bar{\alpha}_{t-1})} \boldsymbol{\varepsilon}' + \sqrt{\beta_t} \boldsymbol{\varepsilon}''}_{\text{两个独立零均值高斯之和}}. \quad (8)$$

由附录 A.1, 两个独立的零均值高斯之和仍是零均值高斯, 其方差为两者方差之和:

$$\alpha_t (1 - \bar{\alpha}_{t-1}) + \beta_t = \alpha_t - \alpha_t \bar{\alpha}_{t-1} + 1 - \alpha_t = 1 - \alpha_t \bar{\alpha}_{t-1} = 1 - \bar{\alpha}_t. \quad (9)$$

又 $\sqrt{\alpha_t \bar{\alpha}_{t-1}} = \sqrt{\bar{\alpha}_t}$, 故 $\mathbf{x}_t$ 服从式 (6)。归纳完成。□

推论 2.4 (起点的分布是已知的) 令 $t = T$ 得 $q(\mathbf{x}_T | \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_T; \sqrt{\bar{\alpha}_T} \mathbf{x}_0, (1 - \bar{\alpha}_T) \mathbf{I})$ 。只要调度使得 $\bar{\alpha}_T \rightarrow 0$, 就有 $\mathbf{x}_T \rightarrow \mathcal{N}(\mathbf{0}, \mathbf{I})$ 且与 $\mathbf{x}_0$ 无关。这保证了反向过程有一个已知的、可以直接采样的起点: 从标准高斯里取一个噪声, 就能开始生成。

2.3 训练时的「想像中」与「实际上」

按定义, 要得到一个含噪样本 $\mathbf{x}_t$ 似乎必须老老实实走 t 步。但命题 2.3 告诉我们可以一步算出。这一点在教学材料中常被专门拿出来强调, 因为它是扩散模型能够高效训练的关键, 也是初学者最容易忽略的地方 [13]。

一句话记住 $\mathbf{x}_t$ 的来源

想像中: $\mathbf{x}_0 \xrightarrow{\sqrt{\beta_1} \boldsymbol{\varepsilon}_1} \mathbf{x}_1 \xrightarrow{\sqrt{\beta_2} \boldsymbol{\varepsilon}_2} \mathbf{x}_2 \cdots \rightarrow \mathbf{x}_t$ (逐步加噪, t 次)

实际上: $\mathbf{x}_0, \boldsymbol{\varepsilon} \xrightarrow{\text{式(7)}} \mathbf{x}_t$ (一次算出, $O(1)$)

训练时随机采一个 t 、再采一个 $\boldsymbol{\varepsilon}$, 直接用闭式解造出 $\mathbf{x}_t$, 并以当初加进去的 $\boldsymbol{\varepsilon}$ 作为监督目标。全程只做一次加噪, 与 t 无关。

2.4 噪声调度与信噪比

$\bar{\alpha}_t$ 随 t 单调递减： t 越大，剩下的信号越少。刻画“信号还剩多少”的常用量是信噪比

$$\text{SNR}(t) = \frac{\bar{\alpha}_t}{1 - \bar{\alpha}_t}, \quad (10)$$

它同样单调递减，从 $t = 1$ 时的约 10^4 一路降到 $t = 1000$ 时的 10^{-5} 量级。 $\{\beta_t\}$ 的取法称为噪声调度，常见两种 [3, 6]：

- 线性调度： β_t 从 $\beta_1 = 10^{-4}$ 线性增到 $\beta_T = 2 \times 10^{-2}$ ， $T = 1000$ 。实现最简单，但两端变化过快、中间过于平坦；
- 余弦调度：令 $\bar{\alpha}_t = \frac{\cos^2\left(\frac{\pi}{2} \cdot \frac{t/T+s}{1+s}\right)}{\cos^2\left(\frac{\pi}{2} \cdot \frac{s}{1+s}\right)}$ ， $s \approx 0.008$ 。它让 $\bar{\alpha}_t$ 的下降更均匀，低噪声阶段保留信号更久，实践中生成质量更好。

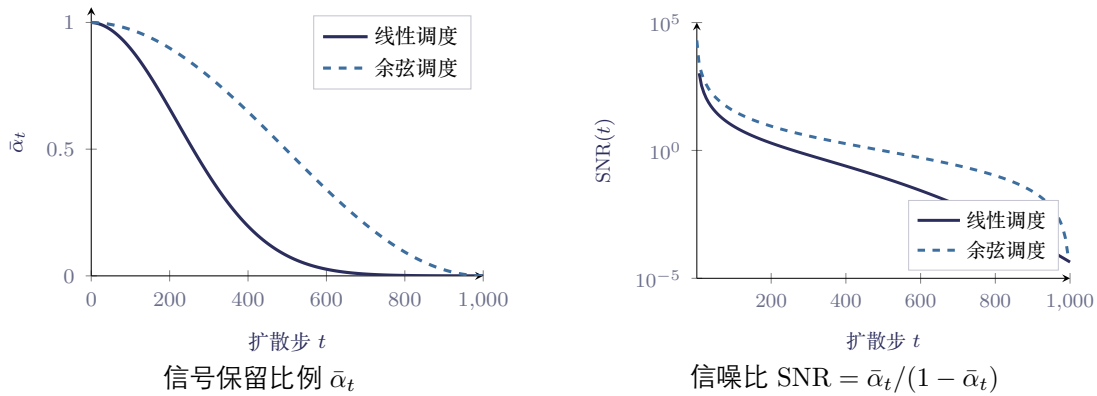


图 2: 两种噪声调度下 $\bar{\alpha}_t$ 与信噪比随扩散步的变化。线性调度的曲线由 $\bar{\alpha}_t \approx \exp(-\sum_{s \leq t} \beta_s)$ 给出

图 2 对比了两种调度。可以看到线性调度在 $t \approx 250$ 时就已经把一半以上的信号换成了噪声，而余弦调度要到 $t \approx 500$ 才降到同一水平。这意味着余弦调度把更多的“容量”留给了中等噪声区间，而这个区间恰恰是反向过程最难学、也最影响生成细节的部分。

3 从 VAE 的似然下界到 DDPM 的下界

3.1 似然算不出来，那就找一个下界

回到式 (3)。以 VAE 为例，模型分布是

$$p_\theta(\mathbf{x}) = \int_{\mathbf{z}} p(\mathbf{z}) p_\theta(\mathbf{x} | \mathbf{z}) d\mathbf{z}, \quad p_\theta(\mathbf{x} | \mathbf{z}) \propto \exp\left(-\frac{1}{2\sigma^2} \|G_\theta(\mathbf{z}) - \mathbf{x}\|^2\right), \quad (11)$$

即解码器输出的是一个均值为 $G_\theta(\mathbf{z})$ 的高斯。这个积分一般没有解析解，也不便于直接最大化。标准的出路是引入一个任意的辅助分布 $q(\mathbf{z} | \mathbf{x})$ （在 VAE 里它就是编码器），再用 Jensen 不等式把对数挪到期望里面去：

$$\begin{aligned} \log p_\theta(\mathbf{x}) &= \log \int_{\mathbf{z}} p_\theta(\mathbf{x}, \mathbf{z}) d\mathbf{z} = \log \int_{\mathbf{z}} q(\mathbf{z} | \mathbf{x}) \frac{p_\theta(\mathbf{x}, \mathbf{z})}{q(\mathbf{z} | \mathbf{x})} d\mathbf{z} \\ &\geq \int_{\mathbf{z}} q(\mathbf{z} | \mathbf{x}) \log \frac{p_\theta(\mathbf{x}, \mathbf{z})}{q(\mathbf{z} | \mathbf{x})} d\mathbf{z} = \mathbb{E}_{q(\mathbf{z} | \mathbf{x})} \left[\log \frac{p_\theta(\mathbf{x}, \mathbf{z})}{q(\mathbf{z} | \mathbf{x})} \right] =: \mathcal{L}(\mathbf{x}), \end{aligned} \quad (12)$$

其中第一步用了 $\log \int = \log \mathbb{E}$ ，第二步用了 Jensen 不等式 $\log \mathbb{E}[\cdot] \geq \mathbb{E}[\log \cdot]$ 。这一项称为证据下界（evidence lower bound, ELBO），取等号当且仅当 $q(\mathbf{z} | \mathbf{x}) = p_\theta(\mathbf{z} | \mathbf{x})$ 。把 $\log \frac{p_\theta(\mathbf{x}, \mathbf{z})}{q(\mathbf{z} | \mathbf{x})}$ 拆开，得到更熟悉的形式

$$\mathcal{L}(\mathbf{x}) = \underbrace{\mathbb{E}_{q(\mathbf{z} | \mathbf{x})} [\log p_\theta(\mathbf{x} | \mathbf{z})]}_{\text{重构项}} - \underbrace{\text{D}_{\text{KL}}(q(\mathbf{z} | \mathbf{x}) \| p(\mathbf{z}))}_{\text{把编码器拉向先验}}. \quad (13)$$

3.2 扩散模型的似然

扩散模型把“一个隐变量 $\mathbf{z}$ ”换成“一串隐变量 $\mathbf{x}_1, \dots, \mathbf{x}_T$ ”，于是

$$\begin{aligned} p_\theta(\mathbf{x}_0) &= \int p(\mathbf{x}_T) \prod_{t=1}^T p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t) d\mathbf{x}_{1:T}, \\ p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t) &\propto \exp\left(-\frac{1}{2\sigma_t^2} \|G_\theta(\mathbf{x}_t) - \mathbf{x}_{t-1}\|^2\right), \end{aligned} \quad (14)$$

形式上与式 (11) 一模一样，只是积分从一个隐变量变成了一串。沿着式 (12) 的路子照抄，把辅助分布取成前向过程 $q(\mathbf{x}_{1:T} | \mathbf{x}_0) = \prod_{t=1}^T q(\mathbf{x}_t | \mathbf{x}_{t-1})$ ，就得到

$$\log p_\theta(\mathbf{x}_0) \geq \mathcal{L}(\mathbf{x}_0) = \mathbb{E}_{q(\mathbf{x}_{1:T} | \mathbf{x}_0)} \left[\log \frac{p_\theta(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T} | \mathbf{x}_0)} \right]. \quad (15)$$

这里有一个非常漂亮的类比 [13]：前向过程就是 VAE 里的编码器。区别在于，这个“编码器”是人为设定、逐层加噪的，不含任何参数、也不需要学习；而“解码器” $p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t)$ 需要学习，且不是一层，而是 T 层。表 3 把两者的对应关系列了出来。

表 3: VAE 与扩散模型的逐项对照

	VAE	扩散模型
隐变量	z , 低维	$\mathbf{x}_1, \dots, \mathbf{x}_T$, 共 T 个, 与数据同维
编码器	$q_\phi(z \mathbf{x})$, 需要学习	$q(\mathbf{x}_{1:T} \mathbf{x}_0)$, 固定、无参数
解码器	$p_\theta(\mathbf{x} z)$, 一层	$p_\theta(\mathbf{x}_{t-1} \mathbf{x}_t)$, T 层, 需学习
训练目标	式 (13) 的 ELBO	式 (15) 的 ELBO
主要代价	结果偏模糊、后验易坍塌	采样需要 T 次网络前向, 慢

3.3 下界的三项分解

把 $p_\theta(\mathbf{x}_{0:T}) = p(\mathbf{x}_T) \prod_{t=1}^T p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t)$ 与 $q(\mathbf{x}_{1:T} | \mathbf{x}_0) = \prod_{t=1}^T q(\mathbf{x}_t | \mathbf{x}_{t-1})$ 代入式 (15), 再对 $t \geq 2$ 的每一对相邻项用贝叶斯公式

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \frac{q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) q(\mathbf{x}_t | \mathbf{x}_0)}{q(\mathbf{x}_{t-1} | \mathbf{x}_0)} \quad (\text{因为 } q(\mathbf{x}_t | \mathbf{x}_{t-1}, \mathbf{x}_0) = q(\mathbf{x}_t | \mathbf{x}_{t-1})), \quad (16)$$

$q(\mathbf{x}_t | \mathbf{x}_0)/q(\mathbf{x}_{t-1} | \mathbf{x}_0)$ 这两项在求和时会相邻相消 (telescoping), 最后只剩首尾两项。整理后得到 DDPM 论文中的经典分解 [3] (完整步骤见附录 A.3):

$$\log p_\theta(\mathbf{x}_0) \geq -(L_T + \sum_{t=2}^T L_{t-1} + L_0), \quad (17)$$

其中三项分别是

$$\begin{cases} L_T = \text{D}_{\text{KL}}(q(\mathbf{x}_T | \mathbf{x}_0) \| p(\mathbf{x}_T)), & \text{常数项, 与 } \theta \text{ 无关;} \\ L_{t-1} = \text{D}_{\text{KL}}(q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) \| p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t)), & \text{主项, 共 } T-1 \text{ 项;} \\ L_0 = -\log p_\theta(\mathbf{x}_0 | \mathbf{x}_1), & \text{重构项.} \end{cases} \quad (18)$$

于是训练扩散模型 = 最小化 $L_T + \sum_{t=2}^T L_{t-1} + L_0$ 。逐项看一眼:

- L_T 只由噪声调度决定, $p(\mathbf{x}_T) = \mathcal{N}(\mathbf{0}, \mathbf{I})$ 是固定的先验, 只要调度让 $\bar{\alpha}_T \rightarrow 0$, $q(\mathbf{x}_T | \mathbf{x}_0)$ 就已经接近 $\mathcal{N}(\mathbf{0}, \mathbf{I})$, 这一项几乎为 0, 所以直接丢掉;
- L_{t-1} 是全部的工作量所在。它在“网络给出的反向分布”与“真实后验”之间算 KL, 而真实后验 $q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0)$ 是一个已知的、可以解析写出的高斯——这正是下一节要用的关键事实;
- L_0 是最后一跳的似然, 实践中通常把它并入 $t=1$ 的那一项, 或对像素做离散化后按 $\{0, \dots, 255\}$ 的分段高斯处理。

本节的落点

L_{t-1} 里的两个分布都是高斯，且方差都可以固定为常数，于是 KL 散度退化成两个均值之间的平方误差。所以“用变分推断推出的下界”最终会变成一个极其朴素的回归问题——这既是扩散模型好训的原因，也是它一开始让人意外的地方。

4 反向过程与训练目标

4.1 反向过程的参数化

反向过程是真正需要学习的部分。它的形式是

$$p_{\theta}(\mathbf{x}_{0:T}) = p(\mathbf{x}_T) \prod_{t=1}^T p_{\theta}(\mathbf{x}_{t-1} | \mathbf{x}_t), \quad p(\mathbf{x}_T) = \mathcal{N}(\mathbf{0}, \mathbf{I}), \quad (19)$$

其中每一步取均值由网络给出、方差固定的高斯：

$$p_{\theta}(\mathbf{x}_{t-1} | \mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_{\theta}(\mathbf{x}_t, t), \sigma_t^2 \mathbf{I}), \quad t = T, T-1, \dots, 1. \quad (20)$$

只让网络决定均值、方差取一个预先定好的 σ_t^2 ，是 DDPM 的关键简化：一方面让后面 L_{t-1} 中的两个高斯方差相同，KL 有干净的闭式；另一方面少了一个学习目标，训练更稳。

注 4.1 (和 VAE 解码器的差别) VAE 的解码器把低维的 $\mathbf{z}$ 映回高维的 $\mathbf{x}$ ，一次跳很远，所以容易模糊；扩散模型的每一步只把 $\mathbf{x}_t$ 挪动一点点到 $\mathbf{x}_{t-1}$ ，输入输出几乎逐像素对应，于是网络实际上是在做“去一点点噪声”这件非常简单的事。 T 步串起来，累计的效果就很强。

4.2 真实后验的闭式解

上一节说过， L_{t-1} 里出现的 $q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0)$ 是已知的。下面把它写出来——这是整篇推导里最需要动手的一块代数。

命题 4.2 (前向过程的后验) 对 $t \geq 2$,

$$q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_{t-1}; \tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0), \tilde{\beta}_t \mathbf{I}), \quad (21)$$

其中

$$\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) = \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t \mathbf{x}_0 + \sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1}) \mathbf{x}_t}{1 - \bar{\alpha}_t}, \quad \tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t. \quad (22)$$

证明 由贝叶斯公式并把 $q(\mathbf{x}_t | \mathbf{x}_{t-1}, \mathbf{x}_0) = q(\mathbf{x}_t | \mathbf{x}_{t-1})$ 代入,

$$q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) = \frac{q(\mathbf{x}_t | \mathbf{x}_{t-1}) q(\mathbf{x}_{t-1} | \mathbf{x}_0)}{q(\mathbf{x}_t | \mathbf{x}_0)}, \quad (23)$$

分母与 $\mathbf{x}_{t-1}$ 无关, 只需看分子。分子的两个因子都是已知高斯:

$$\begin{aligned} q(\mathbf{x}_t | \mathbf{x}_{t-1}) &\propto \exp\left(-\frac{\|\mathbf{x}_t - \sqrt{\alpha_t} \mathbf{x}_{t-1}\|^2}{2\beta_t}\right), \\ q(\mathbf{x}_{t-1} | \mathbf{x}_0) &\propto \exp\left(-\frac{\|\mathbf{x}_{t-1} - \sqrt{\bar{\alpha}_{t-1}} \mathbf{x}_0\|^2}{2(1 - \bar{\alpha}_{t-1})}\right). \end{aligned} \quad (24)$$

把指数相加, 写成关于 $\mathbf{x}_{t-1}$ 的二次型 $-\frac{1}{2}(A\|\mathbf{x}_{t-1}\|^2 - 2\mathbf{x}_{t-1}^\top \mathbf{b}) + C$, 其中

$$\begin{aligned} A &= \frac{\alpha_t}{\beta_t} + \frac{1}{1 - \bar{\alpha}_{t-1}} = \frac{\alpha_t(1 - \bar{\alpha}_{t-1}) + \beta_t}{\beta_t(1 - \bar{\alpha}_{t-1})} = \frac{1 - \bar{\alpha}_t}{\beta_t(1 - \bar{\alpha}_{t-1})}, \\ \mathbf{b} &= \frac{\sqrt{\alpha_t}}{\beta_t} \mathbf{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}}}{1 - \bar{\alpha}_{t-1}} \mathbf{x}_0. \end{aligned} \quad (25)$$

二次型配方即得高斯 $\mathcal{N}(\mathbf{b}/A, A^{-1}\mathbf{I})$ 。代回 $A^{-1} = \frac{\beta_t(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} = \tilde{\beta}_t$, 并整理 $\mathbf{b}/A$ 便得到式 (22)。□

这个命题的意义在于: 它把一条 T 步的链, 压缩成了任意一对相邻状态之间的条件分布。有了它, L_{t-1} 里“真实后验 vs 网络输出”的对比就变成了两个显式高斯之间的对比。

4.3 从“预测均值”到“预测噪声”

有了命题 4.2, 就可以把 L_{t-1} 写开了。取 $\sigma_t^2 = \beta_t$, 由附录 A.2 的高斯 KL 公式 (两者方差相同, 只有均值不同):

$$L_{t-1} = \mathbb{E}_{q(\mathbf{x}_t | \mathbf{x}_0)} \left[\text{D}_{\text{KL}}(\mathcal{N}(\tilde{\boldsymbol{\mu}}_t, \beta_t \mathbf{I}) \parallel \mathcal{N}(\boldsymbol{\mu}_\theta, \beta_t \mathbf{I})) \right] = \mathbb{E}_{q(\mathbf{x}_t | \mathbf{x}_0)} \left[\frac{1}{2\beta_t} \|\tilde{\boldsymbol{\mu}}_t - \boldsymbol{\mu}_\theta(\mathbf{x}_t, t)\|^2 \right]. \quad (26)$$

于是最小化 L_{t-1} , 就是让网络输出的均值去逼近 $\tilde{\boldsymbol{\mu}}_t$ 。

问题是 $\tilde{\boldsymbol{\mu}}_t$ 里含 $\mathbf{x}_0$, 而 $\mathbf{x}_0$ 恰好是我们想生成的东西, 采样时根本拿不到。

出路是利用重参数化把 $\mathbf{x}_0$ 解出来：

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\varepsilon} \implies \mathbf{x}_0 = \frac{\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\varepsilon}}{\sqrt{\bar{\alpha}_t}}. \quad (27)$$

代回式 (22) 中的 $\tilde{\boldsymbol{\mu}}_t$ 并整理，得到一个只含 $\mathbf{x}_t$ 与 $\boldsymbol{\varepsilon}$ 的漂亮形式

$$\boxed{\tilde{\boldsymbol{\mu}}_t = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\varepsilon} \right)}. \quad (28)$$

换言之：“这一步该往哪走”完全由“这一步被加了什么噪声”决定，不需要知道 $\mathbf{x}_0$ 。既然 $\boldsymbol{\varepsilon}$ 未知，就干脆让网络直接去预测它。取

$$\boldsymbol{\mu}_\theta(\mathbf{x}_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\varepsilon}_\theta(\mathbf{x}_t, t) \right), \quad (29)$$

代回式 (26)，两个均值之差变成

$$\|\tilde{\boldsymbol{\mu}}_t - \boldsymbol{\mu}_\theta\|^2 = \frac{\beta_t^2}{\alpha_t(1 - \bar{\alpha}_t)} \|\boldsymbol{\varepsilon} - \boldsymbol{\varepsilon}_\theta(\mathbf{x}_t, t)\|^2, \quad (30)$$

于是

$$L_{t-1} = w_t \mathbb{E} \left[\|\boldsymbol{\varepsilon} - \boldsymbol{\varepsilon}_\theta(\mathbf{x}_t, t)\|^2 \right], \quad w_t = \frac{\beta_t^2}{2\sigma_t^2 \alpha_t (1 - \bar{\alpha}_t)}. \quad (31)$$

注 4.3 (式 (28) 的验算) 把 $\boldsymbol{\varepsilon} = (\mathbf{x}_t - \sqrt{\bar{\alpha}_t} \mathbf{x}_0) / \sqrt{1 - \bar{\alpha}_t}$ 代进去：

$$\frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{1 - \bar{\alpha}_t} (\mathbf{x}_t - \sqrt{\bar{\alpha}_t} \mathbf{x}_0) \right) = \frac{\sqrt{\bar{\alpha}_t} (1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} \mathbf{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}} \beta_t}{1 - \bar{\alpha}_t} \mathbf{x}_0, \quad (32)$$

最后恰好是式 (22) 中的 $\tilde{\boldsymbol{\mu}}_t$ 。

4.4 简化损失函数

最后一步：丢掉权重 w_t ，并对所有 t 取简单平均。DDPM 论文指出，去掉 w_t 相当于给不同噪声等级的下界项重新加权，实验上反而能得到更好的生成质量 [3]。这样就得到了扩散模型最著名的那个目标函数：

$$\boxed{\mathcal{L}_{\text{simple}}(\theta) = \mathbb{E}_{\substack{t \sim \mathcal{U}\{1, \dots, T\} \\ \mathbf{x}_0 \sim p_{\text{data}}, \boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})}} \left[\|\boldsymbol{\varepsilon} - \boldsymbol{\varepsilon}_\theta(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\varepsilon}, t)\|^2 \right]}. \quad (33)$$

读一遍这个式子，它的内容朴素得有点出人意料：

1. 取一张干净数据 $\mathbf{x}_0$;
2. 随机采一个噪声等级 t , 再采一撮标准高斯噪声 $\boldsymbol{\varepsilon}$;
3. 按闭式解合成含噪图 $\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\varepsilon}$;
4. 把 $\mathbf{x}_t$ 和 t 喂给网络, 让它猜出刚才那撮噪声 $\boldsymbol{\varepsilon}$;
5. 用平方误差回传梯度。

一个由变分推断严格推出的下界, 最后化简成了“看图猜噪声”这样一个回归任务。它形式上就是一个去噪自编码器 (denoising autoencoder) 的目标, 也与去噪得分匹配 (denoising score matching) 等价 [11, 12]。

预测噪声 = 估计得分

由 $q(\mathbf{x}_t | \mathbf{x}_0)$ 的高斯形式可以直接求对数密度的梯度:

$$\nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t | \mathbf{x}_0) = -\frac{\mathbf{x}_t - \sqrt{\bar{\alpha}_t} \mathbf{x}_0}{1 - \bar{\alpha}_t} = -\frac{\boldsymbol{\varepsilon}}{\sqrt{1 - \bar{\alpha}_t}}.$$

边际得分 $\nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t) = \mathbb{E}[\nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t | \mathbf{x}_0)]$, 所以“预测噪声 $\boldsymbol{\varepsilon}_\theta$ ”与“估计数据分布的对数密度梯度”是同一件事的两种说法。这也解释了为什么扩散模型可以写成随机微分方程、并用概率流 ODE 来采样。

4.5 三种常用的参数化

式 (27) 说明 $\mathbf{x}_0$ 、 $\mathbf{x}_t$ 、 $\boldsymbol{\varepsilon}$ 三个量之间两两可以互推, 所以网络输出哪一个目标都可以, 差别在于不同噪声等级下的数值稳定性。常见三种:

表 4: 三种参数化方式及其换算关系

参数化	网络输出	反解	特点
$\boldsymbol{\varepsilon}$ -参数化	$\boldsymbol{\varepsilon}$	$\mathbf{x}_0 = \frac{\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\varepsilon}}{\sqrt{\bar{\alpha}_t}}$	高噪声区间稳定, DDPM 默认
$\mathbf{x}_0$ -参数化	$\mathbf{x}_0$	$\boldsymbol{\varepsilon} = \frac{\mathbf{x}_t - \sqrt{\bar{\alpha}_t} \mathbf{x}_0}{\sqrt{1 - \bar{\alpha}_t}}$	低噪声区间清晰, 高噪声易发散
$\mathbf{v}$ -参数化	$\mathbf{v} = \sqrt{\bar{\alpha}_t} \boldsymbol{\varepsilon} - \sqrt{1 - \bar{\alpha}_t} \mathbf{x}_0$	$\boldsymbol{\varepsilon} = \sqrt{\bar{\alpha}_t} \mathbf{v} + \sqrt{1 - \bar{\alpha}_t} \mathbf{x}_t$	全区间数值稳定, 训练超参更宽容

三种参数化在数学上完全等价、可以逐点互相换算, 但经验风险关于 θ 的加权方式不同: $\boldsymbol{\varepsilon}$ -参数化对大 t (高噪声) 的误差更敏感, $\mathbf{x}_0$ -参数化对小 t 更敏感, $\mathbf{v}$ -参数化相当于两端折中, 因此在 T 很大或精度要求较高时常常更省事 [8]。

5 训练与采样算法

5.1 训练

式 (33) 已经把所有东西都准备好了，训练过程也就没有任何魔法：

算法 1 扩散模型的训练（对应式 (33)）

输入： 训练集 $\{\mathbf{x}_0^{(n)}\} \sim p_{\text{data}}$ ，噪声调度 $\{\beta_t\}_{t=1}^T$ ，网络 ε_θ

- 1: **repeat**
- 2: 采样一个干净样本 $\mathbf{x}_0 \sim p_{\text{data}}$
- 3: 采样一个噪声等级 $t \sim \mathcal{U}\{1, \dots, T\}$
- 4: 采样一撮噪声 $\varepsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
- 5: 由闭式解合成含噪样本 $\mathbf{x}_t = \sqrt{\alpha_t} \mathbf{x}_0 + \sqrt{1 - \alpha_t} \varepsilon$
- 6: 取梯度下降步，梯度为 $\nabla_\theta \|\varepsilon - \varepsilon_\theta(\mathbf{x}_t, t)\|^2$
- 7: **until** 收敛

注意算法 1 的一个特点：每一轮只加一次噪。因为有了闭式解， t 可以任意跳，不需要按顺序走完整条链，所以每一步的计算代价与 T 无关。这是扩散模型训练成本其实并不高的原因。

5.2 采样

采样则是反向走完 T 步：

算法 2 扩散模型的采样（DDPM， T 步）

输入： 训练好的网络 ε_θ ，噪声调度 $\{\beta_t\}$ ，反向方差 σ_t^2

输出： 一个生成样本 $\mathbf{x}_0$

- 1: 采样起点 $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
- 2: **for** $t = T, T-1, \dots, 1$ **do**
- 3: **if** $t > 1$ **then**
- 4: 采样 $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
- 5: **else**
- 6: $\mathbf{z} \leftarrow \mathbf{0}$
- 7: **end if**
- 8: $\mathbf{x}_{t-1} \leftarrow \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \alpha_t}} \varepsilon_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$
- 9: **end for**
- 10: **return** $\mathbf{x}_0$

算法 2 的结构与算法 1 高度对称：训练是“把已知的噪声加进去”再让网络把它猜出来，采样是“把预测出来的噪声减掉”。把两段伪代码并排看一遍，基本就理解扩散模型的全部机制了。

5.3 反向方差 σ_t^2 的取法：随机采样与确定性采样

DDPM 论文给出两种取法 [3]:

$$\sigma_t^2 = \beta_t \quad \text{或} \quad \sigma_t^2 = \tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t. \quad (34)$$

前者在 $\bar{\alpha}_t$ 变化平缓时与后者接近；后者在 $t = 1$ 时恰好为 0，意味着最后一步不加噪声。 σ_t 实际上是一个“随机性旋钮”：取正数，采样是随机的；取 $\sigma_t = 0$ ，反向链就退化成一个确定性的映射，同一个 $\mathbf{x}_T$ 永远产生同一个输出。

注 5.1 (DDIM: 把 1000 步压到 50 步) DDIM 从另一个角度出发 [10]: 它构造了一个非马尔可夫的前向过程，其边缘分布 $q(\mathbf{x}_t | \mathbf{x}_0)$ 与 DDPM 完全相同，因此可以复用同一个训练好的网络，但采样式改写为“先估计 $\mathbf{x}_0$ 、再重新构造 $\mathbf{x}_{t-1}$ ”:

$$\hat{\mathbf{x}}_0 = \frac{\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)}{\sqrt{\bar{\alpha}_t}}, \quad \mathbf{x}_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \hat{\mathbf{x}}_0 + \sqrt{1 - \bar{\alpha}_{t-1}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t). \quad (35)$$

因为前向过程不再要求马尔可夫性， $\{\mathbf{x}_t\}$ 可以只在任意子序列 $\tau_1 < \dots < \tau_S$ 上取值。于是 1000 步可以被压缩到 20 ~ 50 步，且同一个模型可以随意改变采样步数而无需重新训练。式 (35) 与“取 $\sigma_t = 0$ ”在形态上很接近（都没有再注入噪声），但 DDIM 的跳步能力来自它的非马尔可夫结构，这一点是单纯的 $\sigma_t = 0$ 做不到的。

5.4 为什么每一步要采样，而不是直接取均值？

既然网络已经给出了均值 $\boldsymbol{\mu}_\theta$ ，每一步直接取均值不是更省事吗？这是学习扩散模型时很自然的一个疑问，它的答案有两层。

第一层是概率上的。 $p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t) = \mathcal{N}(\boldsymbol{\mu}_\theta, \sigma_t^2 \mathbf{I})$ 本身是一个分布，均值只是它的中心。要“从分布中生成样本”，就必须从分布里采样；若每一步都取均值，整条反向链变成一个确定性函数 $\mathbf{x}_0 = f(\mathbf{x}_T)$ ，同一个起点永远给出同一张图，众数唯一，样本多样性随之丧失。

第二层是生成任务的实践经验。在序列生成（文本、语音）中，“每一步都取概率最大的那个词”会得到重复、单调甚至崩塌的结果，这一现象早在语言模型采样中就有系统讨论（*The Curious Case of Neural Text Degeneration*[5]）；语音合成中也观察到类似现象：完全去掉采样（例如去掉 dropout 带来的随机性）会让韵律变得机械、不自然 [13]。

反过来说，这两点也说明均值已经携带了绝大部分信息。把 σ_t 设成 0，生

成的图像依然可以接受、甚至更锐利，说明噪声更多影响的是多样性与细节的随机性，而不是图像的主体结构。所以 σ_t 更像是一个“多样性 $\leftrightarrow$ 确定性”的旋钮，而不是一个非此即彼的选择。

5.5 一个最小可运行的实现

把上面两段伪代码直译成 PyTorch，核心不到 40 行（完整可运行版本见 `code/ddpm_minimal.py`）：

Listing 1: DDPM 的最小实现（训练与采样）

```

1  import torch
2
3  T = 1000
4  beta = torch.linspace(1e-4, 2e-2, T)      # linear noise schedule
5  alpha = 1.0 - beta
6  abar = torch.cumprod(alpha, dim=0)         # cumulative product of alpha
7
8  def q_sample(x0, t, noise):
9      """Forward closed form:  $x_t = \sqrt{abar_t} x_0 + \sqrt{1-abar_t} \text{ noise}$ 
10      """
11      a = abar[t].sqrt().view(-1, *[1] * (x0.dim() - 1))
12      s = (1.0 - abar[t]).sqrt().view(-1, *[1] * (x0.dim() - 1))
13      return a * x0 + s * noise
14
15  def train_step(model, x0, opt):
16      t = torch.randint(0, T, (x0.shape[0],), device=x0.device)
17      noise = torch.randn_like(x0)
18      xt = q_sample(x0, t, noise)             # one shot, 0(1)
19      loss = ((model(xt, t) - noise) ** 2).mean() # predict the noise
20      opt.zero_grad(); loss.backward(); opt.step()
21      return loss.item()
22
23  @torch.no_grad()
24  def sample(model, shape):
25      x = torch.randn(shape)                  #  $x_T \sim N(0, I)$ 
26      for ti in reversed(range(T)):
27          t = torch.full((shape[0],), ti, dtype=torch.long)
28          eps = model(x, t)
29          mean = (x - beta[ti] / (1.0 - abar[ti]).sqrt() * eps) / alpha[ti].sqrt()

```

```

29         if ti > 0:
30             x = mean + beta[ti].sqrt() * torch.randn_like(x)  # sigma_t
           ^2 = beta_t
31         else:
32             x = mean
33     return x

```

6 延伸与应用

6.1 连续化的视角：随机微分方程

把 T 取得很大、把 β_t 取得很小的极限下，前向过程就是一个随机微分方程 (SDE)

$$d\mathbf{x} = -\frac{1}{2}\beta(t)\mathbf{x}dt + \sqrt{\beta(t)}d\mathbf{w}, \quad (36)$$

其中 $\mathbf{w}$ 是标准布朗运动。它的反向时间 SDE 为

$$d\mathbf{x} = \left[-\frac{1}{2}\beta(t)\mathbf{x} - \beta(t)\nabla_{\mathbf{x}}\log q_t(\mathbf{x}) \right]dt + \sqrt{\beta(t)}d\bar{\mathbf{w}}, \quad (37)$$

即只需要知道每个时刻的得分函数 $\nabla_{\mathbf{x}}\log q_t(\mathbf{x})$ ；而由上一节的结论，得分恰好由噪声预测网络给出。Song 等人指出，同一族前向 SDE 都对应一个概率流 ODE

$$d\mathbf{x} = \left[-\frac{1}{2}\beta(t)\mathbf{x} - \frac{1}{2}\beta(t)\nabla_{\mathbf{x}}\log q_t(\mathbf{x}) \right]dt, \quad (38)$$

沿着它积分能得到完全确定性的采样轨迹，且与 SDE 采样共享同一组边缘分布 [11]。这从理论上解释了 DDIM 的确定性采样与跳步能力：它正是概率流 ODE 的一种数值离散。

6.2 语音与文本

扩散模型最初在图像上取得突破，随后被搬到其他模态，而不同模态的难点恰好揭示了它的边界。

语音几乎没有障碍，因为语音波形本身是连续的高维实值信号，与图像的设置同构。WaveGrad、DiffWave 等模型把扩散用于声码器与声学模型，用几十步采样即可合成高质量语音 [1]。

文本则要麻烦得多，原因是文本是离散的：标准高斯噪声无法与 token 的 one-hot 表示直接相加，而前向过程一旦不再可加高斯，闭式解与整套推导都会

失效。实践中形成了三条路：

- 在隐空间上加噪：先用一个连续的自编码器把文本（或图像）压到连续的隐空间，再在隐空间上做标准扩散。Latent Diffusion 在图像上非常成功，DiffuSeq 等把这一思路搬到文本，可同时生成一对相关的句子 [7]；
- 换掉高斯噪声：不用高斯噪声，而用“编辑”（插入、删除、替换）这类离散的扰动来定义前向过程，DiffusER 是这一路线的代表；
- 用掩码代替加噪：把“加噪—去噪”换成“加掩码—预测”，即 Mask-Predict 一类的非自回归生成方法，可以并行地一次填空若干位置 [2]。

注 6.1 (一个统一的理解) 如果把“扩散”理解为把一次生成拆成若干次微小的、逐步确定化的生成，那么这些变体其实是同一件事在不同数据模态上的实现：连续数据用高斯噪声，离散数据用掩码或编辑操作，维度太高（例如高分辨率图像）就先压到隐空间再扩散。

6.3 条件生成与工程实践

要控制生成结果（“一只在奔跑的狗”），只需把条件 c 交给网络： $\epsilon_\theta(\mathbf{x}_t, t) \rightarrow \epsilon_\theta(\mathbf{x}_t, t, c)$ ，训练时把 c （文本嵌入、类别标签）与时间步 t 一起注入。更细的控制则通过引导（guidance）实现：

- 分类器引导（classifier guidance）：额外训一个分类器 $p_\phi(c | \mathbf{x}_t)$ ，在采样时把它的梯度加到均值上， $\tilde{\mu} = \mu_\theta + \sigma_t^2 \nabla_{\mathbf{x}_t} \log p_\phi(c | \mathbf{x}_t)$ ；
- 无分类器引导（classifier-free guidance）：不训分类器，而是同时训练“有条件”与“无条件”两种预测，采样时做外推 $\tilde{\epsilon} = (1 + w) \epsilon_\theta(\mathbf{x}_t, t, c) - w \epsilon_\theta(\mathbf{x}_t, t, \emptyset)$ ， w 越大越贴合条件、但多样性越低 [4]。

最后是工程上最常被提到的三个议题：

- 采样太慢： T 次网络前向是主要成本。加速思路有三类——改采样器（DDIM、DPM-Solver 等把步数压到几十步甚至十几步）、蒸馏（Progressive Distillation、一致性模型，把多步压成 1~4 步）、在隐空间扩散（把每次前向的分辨率降下来）；
- 损失加权：式 (31) 中的 w_t 丢掉之后虽然效果好，但“如何加权”本身是一个可以调的设计空间， ϵ/v 参数化与 Min-SNR 加权都属于这一类；
- 训练数据与算力：扩散模型的训练是稳定的（没有 GAN 那样的对抗失衡），代价转移到数据规模与采样时间上，这也是它在这几年迅速工业化的原因之一。

7 小结

把整篇内容压成一条线，扩散模型的逻辑其实非常顺：

1. 目标：生成模型想学 $p_{\text{data}}(\mathbf{x})$ ，最大似然等价于最小化 $D_{\text{KL}}(p_{\text{data}}\|p_{\theta})$ ，但 $p_{\theta}(\mathbf{x})$ 里的积分算不出来；
2. 方案：把“一步生成”改成“ T 步微小的去噪”。用一条固定、无参数的前向链把数据逐步溶成噪声（相当于 VAE 的编码器），再学一条反向链把它还原（解码器）；
3. 前向闭式解： $q(\mathbf{x}_t | \mathbf{x}_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}\mathbf{x}_0, (1 - \bar{\alpha}_t)\mathbf{I})$ 让 $\mathbf{x}_t$ 可以一步造出来，训练代价与 T 无关；
4. 下界：由 Jensen 不等式得 ELBO，展开成 $L_T + \sum_{t \geq 2} L_{t-1} + L_0$ 。其中 L_T 是常数， L_{t-1} 是主项，而真实后验 $q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0)$ 有闭式解；
5. 化简：两个方差相同的正态分布之间的 KL 退化成均值平方误差，再把均值换成“预测噪声”的写法，就得到 $\mathcal{L}_{\text{simple}} = \mathbb{E}\|\boldsymbol{\varepsilon} - \boldsymbol{\varepsilon}_{\theta}(\mathbf{x}_t, t)\|^2$ ；
6. 采样：从 $\mathcal{N}(\mathbf{0}, \mathbf{I})$ 出发反向走 T 步，每步减掉预测的噪声、再按 σ_t 补一点新噪声。 σ_t 控制随机性； $\sigma_t \rightarrow 0$ 得到确定性采样，DDIM 由此把步数压到几十步；
7. 联系：预测噪声等价于估计得分函数，于是扩散模型可以写成反向时间 SDE 与概率流 ODE，也解释了它与 VAE、得分匹配之间的关系。

一句话

扩散模型 = 用一条固定的加噪链，把一个不可解的积分问题，换成了一个可以解析写出下界的、逐层的去噪回归问题。它的所有“魔法”都来自前向过程的闭式解 $q(\mathbf{x}_t | \mathbf{x}_0)$ ：正因为 $\mathbf{x}_t$ 可以一步造出来，训练才会既简单又便宜；也正因为真实后验可以解析写出，变分下界才能化简成“看图猜噪声”。

表 5 是几个核心公式的速查表，动手实现时可以据此逐行对照。

A 两个高斯分布的常用结论

A.1 独立高斯之和仍是高斯

命题 A.1 设 $\boldsymbol{\varepsilon}_1 \sim \mathcal{N}(\mathbf{0}, \sigma_1^2 \mathbf{I})$ 、 $\boldsymbol{\varepsilon}_2 \sim \mathcal{N}(\mathbf{0}, \sigma_2^2 \mathbf{I})$ 相互独立， $a, b \in \mathbb{R}$ ，则

$$a\boldsymbol{\varepsilon}_1 + b\boldsymbol{\varepsilon}_2 \sim \mathcal{N}(\mathbf{0}, (a^2\sigma_1^2 + b^2\sigma_2^2)\mathbf{I}). \quad (39)$$

表 5: 核心公式速查

环节	公式
前向单步	$\mathbf{x}_t = \sqrt{\alpha_t} \mathbf{x}_{t-1} + \sqrt{\beta_t} \boldsymbol{\varepsilon}$
前向闭式	$\mathbf{x}_t = \sqrt{\alpha_t} \mathbf{x}_0 + \sqrt{1 - \alpha_t} \boldsymbol{\varepsilon}$
后验均值	$\tilde{\boldsymbol{\mu}}_t = \frac{\sqrt{\alpha_{t-1}} \beta_t \mathbf{x}_0 + \sqrt{\alpha_t} (1 - \alpha_{t-1}) \mathbf{x}_t}{1 - \alpha_t}$
训练目标	$\mathcal{L}_{\text{simple}} = \mathbb{E} \ \boldsymbol{\varepsilon} - \boldsymbol{\varepsilon}_\theta(\mathbf{x}_t, t)\ ^2$
采样单步	$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \alpha_t}} \boldsymbol{\varepsilon}_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$
从噪声还原数据	$\hat{\mathbf{x}}_0 = \frac{\mathbf{x}_t - \sqrt{1 - \alpha_t} \boldsymbol{\varepsilon}_\theta(\mathbf{x}_t, t)}{\sqrt{\alpha_t}}$
噪声与得分	$\nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t) = -\frac{\boldsymbol{\varepsilon}_\theta(\mathbf{x}_t, t)}{\sqrt{1 - \alpha_t}}$

证明 设 $\mathbf{t} \in \mathbb{R}^d$, 用特征函数。由于 $\boldsymbol{\varepsilon}_1, \boldsymbol{\varepsilon}_2$ 独立,

$$\mathbb{E}[e^{i\mathbf{t}^\top (a\boldsymbol{\varepsilon}_1 + b\boldsymbol{\varepsilon}_2)}] = \mathbb{E}[e^{iat^\top \boldsymbol{\varepsilon}_1}] \cdot \mathbb{E}[e^{ibt^\top \boldsymbol{\varepsilon}_2}] = e^{-\frac{1}{2}a^2\sigma_1^2\|\mathbf{t}\|^2} \cdot e^{-\frac{1}{2}b^2\sigma_2^2\|\mathbf{t}\|^2} = e^{-\frac{1}{2}(a^2\sigma_1^2 + b^2\sigma_2^2)\|\mathbf{t}\|^2}, \quad (40)$$

这正是零均值高斯 $\mathcal{N}(\mathbf{0}, (a^2\sigma_1^2 + b^2\sigma_2^2)\mathbf{I})$ 的特征函数, 由特征函数与分布的一一对应即得结论。□

这个结论在正文中被用了两次: 一次是在前向闭式解的归纳法里 (式 (8)), 把两步噪声合并成一步; 另一次是在推导 $\tilde{\boldsymbol{\mu}}_t$ 时把“关于 $\mathbf{x}_{t-1}$ 的二次型”配方成高斯。

A.2 两个高斯之间的 KL 散度

命题 A.2 设 $\boldsymbol{\mu}_1, \boldsymbol{\mu}_2 \in \mathbb{R}^d$, $\boldsymbol{\Sigma}_1, \boldsymbol{\Sigma}_2$ 为 $d \times d$ 正定矩阵, 则

$$\begin{aligned} D_{\text{KL}}(\mathcal{N}(\boldsymbol{\mu}_1, \boldsymbol{\Sigma}_1) \parallel \mathcal{N}(\boldsymbol{\mu}_2, \boldsymbol{\Sigma}_2)) &= \frac{1}{2} \left[\text{tr}(\boldsymbol{\Sigma}_2^{-1} \boldsymbol{\Sigma}_1) + (\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)^\top \boldsymbol{\Sigma}_2^{-1} (\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1) \right. \\ &\quad \left. - d + \ln \frac{\det \boldsymbol{\Sigma}_2}{\det \boldsymbol{\Sigma}_1} \right]. \end{aligned} \quad (41)$$

特别地, 当两者协方差相同 ($\boldsymbol{\Sigma}_1 = \boldsymbol{\Sigma}_2 = \sigma^2 \mathbf{I}$) 时, 只有均值不同, 公式退化成

$$D_{\text{KL}}(\mathcal{N}(\boldsymbol{\mu}_1, \sigma^2 \mathbf{I}) \parallel \mathcal{N}(\boldsymbol{\mu}_2, \sigma^2 \mathbf{I})) = \frac{1}{2\sigma^2} \|\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2\|^2. \quad (42)$$

式 (42) 就是正文式 (26) 的来源: 它把“两个分布有多远”这件事, 换成了“两

个均值差多少”这件更简单的事，而后者正好可以用平方误差来优化。

A.3 ELBO 三项分解的完整展开

从式 (15) 出发，把两个链式乘积代入：

$$\begin{aligned}
 \mathcal{L}(\mathbf{x}_0) &= \mathbb{E}_q \left[\log p(\mathbf{x}_T) + \sum_{t=1}^T \log p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t) - \sum_{t=1}^T \log q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) \right] \\
 &= \mathbb{E}_q \left[\log p(\mathbf{x}_T) + \log p_\theta(\mathbf{x}_0 \mid \mathbf{x}_1) \right. \\
 &\quad \left. + \sum_{t=2}^T \log p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t) - \sum_{t=2}^T \log q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) - \log q(\mathbf{x}_1 \mid \mathbf{x}_0) \right].
 \end{aligned} \tag{43}$$

对 $t \geq 2$ 的每一对相邻项用贝叶斯公式 (16)：

$$\begin{aligned}
 \sum_{t=2}^T \log q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) &= \sum_{t=2}^T \log q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) \\
 &\quad + \underbrace{\sum_{t=2}^T [\log q(\mathbf{x}_t \mid \mathbf{x}_0) - \log q(\mathbf{x}_{t-1} \mid \mathbf{x}_0)]}_{\text{相消}},
 \end{aligned} \tag{44}$$

后一个求和是相邻两项的差，只剩首尾： $\log q(\mathbf{x}_T \mid \mathbf{x}_0) - \log q(\mathbf{x}_1 \mid \mathbf{x}_0)$ 。代回式 (43)，其中 $-\log q(\mathbf{x}_1 \mid \mathbf{x}_0)$ 恰好被抵消，于是

$$\mathcal{L}(\mathbf{x}_0) = \mathbb{E}_q \left[\log p_\theta(\mathbf{x}_0 \mid \mathbf{x}_1) - \sum_{t=2}^T \log \frac{q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0)}{p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t)} - \log \frac{q(\mathbf{x}_T \mid \mathbf{x}_0)}{p(\mathbf{x}_T)} \right]. \tag{45}$$

现在逐项看：对固定的 t ，第二项里除 $\mathbf{x}_t$ 之外的变量都能积掉，期望退化为 $\mathbb{E}_{q(\mathbf{x}_t \mid \mathbf{x}_0)} [\text{D}_{\text{KL}}(q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) \parallel p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t))]$ ；第三项就是 $\text{D}_{\text{KL}}(q(\mathbf{x}_T \mid \mathbf{x}_0) \parallel p(\mathbf{x}_T))$ ；第一项是 $-\log p_\theta(\mathbf{x}_0 \mid \mathbf{x}_1)$ 的期望。于是

$$\begin{aligned}
 \mathcal{L}(\mathbf{x}_0) &= \underbrace{-\text{D}_{\text{KL}}(q(\mathbf{x}_T \mid \mathbf{x}_0) \parallel p(\mathbf{x}_T))}_{L_T} \\
 &\quad - \sum_{t=2}^T \underbrace{\mathbb{E}_{q(\mathbf{x}_t \mid \mathbf{x}_0)} [\text{D}_{\text{KL}}(q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) \parallel p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t))]}_{L_{t-1}} \\
 &\quad - \underbrace{\mathbb{E}[-\log p_\theta(\mathbf{x}_0 \mid \mathbf{x}_1)]}_{L_0},
 \end{aligned} \tag{46}$$

即 $\mathcal{L}(\mathbf{x}_0) = -(L_T + \sum_{t=2}^T L_{t-1} + L_0)$, 与正文式 (17) 一致。

参考文献

- [1] N. Chen, Y. Zhang, H. Zen, R. J. Weiss, M. Norouzi, and W. Chan. *WaveGrad: Estimating Gradients for Waveform Generation*. 2021. arXiv: [2009.00713 \[eess.AS\]](#).
- [2] M. Ghazvininejad, O. Levy, Y. Liu, and L. Zettlemoyer. “Mask-Predict: Parallel Decoding of Conditional Masked Language Models”. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2019, pp. 6112–6121.
- [3] J. Ho, A. Jain, and P. Abbeel. *Denoising Diffusion Probabilistic Models*. 2020. arXiv: [2006.11239 \[cs.LG\]](#).
- [4] J. Ho and T. Salimans. “Classifier-Free Diffusion Guidance”. In: *NeurIPS 2021 Workshop on Deep Generative Models and Downstream Applications*. 2022.
- [5] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi. “The Curious Case of Neural Text Degeneration”. In: *International Conference on Learning Representations (ICLR)*. 2020.
- [6] A. Q. Nichol and P. Dhariwal. *Improved Denoising Diffusion Probabilistic Models*. 2021. arXiv: [2102.09672 \[cs.LG\]](#).
- [7] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer. “High-Resolution Image Synthesis with Latent Diffusion Models”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2022, pp. 10684–10695.
- [8] T. Salimans and J. Ho. *Progressive Distillation for Fast Sampling of Diffusion Models*. 2022. arXiv: [2202.00512 \[cs.LG\]](#).
- [9] J. Sohl-Dickstein, E. A. Weiss, N. Maheswaranathan, and S. Ganguli. “Deep Unsupervised Learning using Nonequilibrium Thermodynamics”. In: *Proceedings of the 32nd International Conference on Machine Learning (ICML)*. 2015, pp. 2256–2265.
- [10] J. Song, C. Meng, and S. Ermon. *Denoising Diffusion Implicit Models*. 2021. arXiv: [2010.02502 \[cs.LG\]](#).

- [11] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole. *Score-Based Generative Modeling through Stochastic Differential Equations*. 2021. arXiv: [2011.13456 \[cs.LG\]](#).
- [12] P. Vincent. “A Connection Between Score Matching and Denoising Autoencoders”. In: vol. 23. 7. MIT Press, 2011, pp. 1661–1674.
- [13] 李宏毅. 机器学习课程讲义: *Diffusion Model (DDPM)*. 课程投影片. v7. 2023.